Метод и система пошаговой оптимизации архитектур программных комплексов с контролем нефункциональных требований в замкнутом контуре имитационного моделирования

Дичковский Владимир Андреевич

Аннотация

Проектирование архитектуры распределенных программных комплексов сопряжено с необходимостью одновременного удовлетворения функциональных сценариев и строгих нефункциональных требований (SLA по задержке, предельному бюджету инфраструктуры, пропускной способности). Применение больших языковых моделей (LLM) для прямой генерации архитектур страдает от галлюцинаций, нарушения архитектурных паттернов и деградации синтаксиса при строгом структурированном выводе. В данной работе предложен метод и программный комплекс пошагового проектирования и оптимизации топологии микросервисных архитектур в замкнутом контуре с обратной связью. Архитектурный синтез разделен на инициализирующую генерацию и итеративные шаги целенаправленного исправления (repair loop) с передачей точечного контекста ошибок схемы и доменных нарушений. В основу верификации заложена расширяемая цепочка валидаторов, включающая строгую структурную онтологию графа (проверка DAG, связности, целостности и матрицы допустимых межсервисных переходов) и математический контроль нефункциональных ограничений. Реализована автоматическая трансляция синтезированного графа в нотацию C4 Container (PlantUML) и выполнена апробация на реалистичных профилях нагрузки.

Ключевые слова: системный дизайн, микросервисная архитектура, большие языковые модели, структурированная генерация, валидация графов, нефункциональные требования, замкнутый контур, теория массового обслуживания.

1 Введение

Проектирование современных распределенных и микросервисных систем является критически важным этапом жизненного цикла разработки ПО. Архитектурные ошибки, допущенные на ранних фазах концептуального проектирования (некорректный выбор протоколов взаимодействия, отсутствие уровней кэширования, циклические блокирующие зависимости или перегрузка баз данных), экспоненциально увеличивают стоимость исправления дефектов на этапе промышленной эксплуатации [1].

В последние годы активно исследуются возможности применения больших языковых моделей (LLM) для автоматизации проектирования программных систем [2, 3]. Однако непосредственная («однопроходная») генерация графов распределенных систем силами LLM сталкивается с фундаментальными барьерами:

1. **Синтаксическая и структурная нестабильность**: генерация строгих форматов (JSON/YAML) подвержена повреждениям структуры данных, выпадению обязательных полей спецификаций емкости компонентов и появлению артефактов рассуждений в выводе.

2. **Архитектурные галлюцинации**: порождение топологически недопустимых связей (например, прямое обращение внешнего неавторизованного клиента к закрытому хранилищу данных) и образование циклов в синхронных путях обработки.

3. **Неспособность детерминированной оценки нефункциональных требований (NFR)**: модели склонны декларативно заявлять об устойчивости предложенного дизайна к нагрузкам без математического расчета вероятностных характеристик задержки, очередей и инфраструктурного бюджета [4].

Для преодоления указанных ограничений необходим переход от чисто эвристической текстовой генерации к *нейросимвольному замкнутому контуру*, в котором генеративный агент взаимодействует с детерминированным конвейером верификации, получая на каждом шаге строгую диагностическую обратную связь.

2 Цель и задачи исследования

Цель исследования: разработка метода, алгоритмического обеспечения и программного комплекса пошаговой оптимизации и синтеза топологий программных комплексов с гарантированным контролем структуры и нефункциональных требований в замкнутом верификационном контуре.

Для достижения поставленной цели сформулированы следующие научно-практические **задачи**:

1. Разработать формальную модель описания архитектурной топологии и систему доменных ограничений (онтологию), регламентирующую допустимые классы компонентов и направленные межкомпонентные интерфейсы взаимодействия.

2. Разработать метод и алгоритмы итеративной генерации архитектуры с разделением фазы первичного синтеза (`INITIAL`) и фазы направленной коррекции (`REPAIR`) с инъекцией контекста синтаксических и структурных ошибок в промпт агента.

3. Спроектировать расширяемый конвейер валидации, объединяющий проверку ацикличности, связности, ссылочной целостности графа и аналитический контроль нефункциональных требований (бюджетные лимиты, задержки, очереди).

4. Реализовать архитектуру программного комплекса `AutoSystemDesign`, модуль автоматического экспорта результатов в нотацию C4 Container (PlantUML) и провести экспериментальное исследование сходимости контура оптимизации на реалистичных профилях нагрузки.

3 Анализ аналогов и предшествующих исследований

Для сопоставления с разрабатываемым подходом рассмотрены четыре группы работ: генерация моделей и архитектурных представлений средствами LLM; мультиагентное проектирование; моделирование производительности микросервисов; аналитические модели

очередей. Сравнение проводится по конкретным функциям: что является входом и выходом метода, как проверяется результат, учитываются ли характеристики нагрузки и может ли результат проверки изменить сгенерированную архитектуру. Такое разделение важно, поскольку генерация диаграммы, имитация уже заданной системы и тестирование развернутого приложения решают разные задачи.

3.1 Генерация моделей и архитектурных артефактов средствами LLM

Подход Diagram-as-Code позволяет хранить архитектурные представления в текстовом виде и воспроизводить диаграммы из исходного описания; книга С. Брауна систематизирует практику визуализации и документирования архитектуры, в том числе с использованием C4 [5]. Однако сам факт представления модели в виде PlantUML, Mermaid или DSL не гарантирует семантической корректности: нотация решает задачу представления модели, но правила допустимости связей и выполнение ограничений производительности должны проверяться отдельно. Обзор Hou et al. систематизирует применение LLM в программной инженерии, но не является отдельным методом синтеза и оптимизации микросервисной топологии [2].

В работе NOMAD предложен мультиагентный конвейер генерации UML-диаграмм классов из текстовых требований: отдельные агенты извлекают сущности, определяют отношения, интегрируют модель в промежуточное JSON-представление и формируют PlantUML; также предусмотрен отдельный агент-валидатор. Авторы оценивают систему на примере Northwind и на наборе заданий, созданных людьми, а ошибки классифицируют как структурные, связанные с отношениями и семантические [6]. Поэтому нельзя утверждать, что предшествующие работы вообще не используют промежуточные представления или проверку. Отличие рассматриваемой задачи заключается в другом объекте проверки: в NOMAD основной артефакт — UML-модель предметной области, тогда как здесь рассматривается топология микросервисов с типами инфраструктурных узлов, допустимыми направленными связями, параметрами пропускной способности, очередей и стоимости. В опубликованном описании NOMAD проверка не представлена как цикл, оптимизирующий топологию по результатам расчета NFR или имитации пикового трафика.

Ferrari et al. исследуют генерацию UML-диаграмм последовательностей из 28 документов с требованиями и отмечают, что полнота и корректность результата ухудшаются, в частности, при неоднозначных или противоречивых требованиях [7]. Эта работа важна как эмпирическое свидетельство того, что синтаксически приемлемый диаграммный артефакт не обязательно точно отражает требования. При этом объект исследования отличается от предлагаемого: авторы анализируют диаграммы последовательностей, а не нагрузочную состоятельность топологии микросервисов.

ArchiGen рассматривает обратное направление преобразования: из UML-диаграмм формируются документы Software Solution Blueprint. Это полезный аналог автоматизации перехода между архитектурными артефактами, но он не совпадает с генерацией топологии из исходных требований и проверкой ее поведения под нагрузкой [8].

Более близки к архитектурному синтезу системы MAAD и C4Modeler. MAAD распределяет проектирование между агентами Analyst, Modeler, Designer и Evaluator и формирует архитектурные чертежи и отчеты об оценке атрибутов качества; в опубликованной работе также приведено сравнение с MetaGPT [9]. C4Modeler использует диалог ролевых агентов для построения представлений C4 Context, Container и Component, сочетая детерминированные проверки структуры и правил C4 с семантической оценкой LLM-as-a-Judge [10]. Следовательно, наличие мультиагентной генерации архитектуры и структурной проверки само по себе не является отличительным признаком настоящего исследования.

Существенный предмет сравнения — включается ли в тот же контур формализованная проверка количественных ограничений по заданному профилю нагрузки и используются ли диагностические результаты для целенаправленного исправления графа. В описаниях MAAD и C4Modeler такой связанный цикл с моделированием очередей или дискретно-событийной симуляцией микросервисов не указан.

3.2 Моделирование производительности и пиковых нагрузок микросервисов

PerfSim — дискретно-событийный симулятор цепочек облачных микросервисов. Он позволяет задавать сценарии, вычислять показатели, включая среднее время ответа, и исследовать различные конфигурации ресурсов и взаимодействий; авторы сообщают об accuracy 81–99% для оценки среднего времени ответа в рассмотренных сценариях [4]. Это уже решает важную часть задачи оценки производительности до проведения полноценного развертывания. При этом PerfSim не является генератором архитектур: его сценарий и модели характеристик микросервисов задаются извне. Следовательно, он может выступать симуляционным оценщиком в контуре проектирования, но не заменяет этап синтеза и исправления топологии.

CloudNativeSim также предназначен для моделирования cloud-native приложений с микросервисной архитектурой. Он поддерживает динамические модели, пользовательские сценарии, интерфейсы политик планирования и оценку QoS; в опубликованном кейсе авторы сообщают о точности выше 94,5% по времени отклика и демонстрируют анализ политик масштабирования [11]. Эти результаты относятся к описанной авторами экспериментальной конфигурации и не должны интерпретироваться как универсальная гарантия точности для произвольной системы. По сравнению с исследуемым подходом CloudNativeSim сосредоточен на моделировании и оценке уже заданного приложения и политик ресурсов, а не на генерации топологии LLM с последующей коррекцией по диагностике модели.

Более ранний и методологически важный пример — Palladio Component Model. Модель задает компоненты и их параметры, контекст использования и характеристики ресурсов, после чего инструмент моделирования рассчитывает показатели производительности для архитектурных вариантов [12]. Работа показывает, что параметрические профили компонентов и симуляция архитектуры задолго до появления LLM уже использовались для поддержки архитектурных решений. Поэтому новизну нельзя формулировать как первое применение моделей производительности к программной архитектуре. В работе Khazaei et al. построена многоуровневая модель производительности микросервисной платформы, применяемая для анализа provisioning и capacity planning, включая сценарии «что будет, если» [13]. Эти исследования дают основу для моделирования ресурсов и системных эффектов, но не описывают LLM-контур генерации и целевого исправления архитектурного графа.

DeathStarBench решает другую, дополняющую задачу: это открытый набор реальных микросервисных приложений и нагрузочных сценариев, предназначенный для исследования сквозного поведения, использования ресурсов и последствий архитектурных решений [14]. В отличие от PerfSim и CloudNativeSim, это прежде всего бенчмарк для экспериментов с развернутыми системами, а не самостоятельный симулятор. Он полезен как эмпирическая база для проверки того, насколько модель производительности соответствует измерениям реального приложения. Сам по себе DeathStarBench не генерирует архитектуру по требованиям и не выполняет автоматический ремонт топологии.

Таким образом, симуляция микросервисов, профили ресурсов и оценка поведения при нагрузке уже являются исследованными направлениями. Предмет настоящего исследования находится не в создании нового симулятора, а в организации единого контура, в

котором генерация архитектурного графа, формализованная проверка NFR и расчетная оценка поведения при нагрузке связаны с диагностикой нарушений и последующим исправлением архитектурного решения. Отличительным свойством в таком случае является не наличие симулятора само по себе, а способ использования его результатов в процедуре выбора и коррекции топологии.

3.3 Мультиагентные системы в программной инженерии и архитектурном проектировании

ChatDev организует разработку ПО как взаимодействие специализированных агентов, выполняющих задачи проектирования, программирования и тестирования, и формализует обмен между ними через последовательности диалогов [15]. MetaGPT переносит в мультиагентный процесс стандартные процедуры разработки и распределяет роли между агентами, стремясь снизить ошибки при последовательной передаче результатов [16]. Эти работы обосновывают использование специализации ролей и промежуточных артефактов, однако их центральная задача — процесс разработки ПО в целом, а не количественная оптимизация топологии микросервисов при ограничениях задержки, пропускной способности и стоимости.

Обзор Liu et al. классифицирует исследования LLM-агентов для программной инженерии по задачам разработки и свойствам агентов, а также описывает открытые проблемы направления [3]. На основании обзора некорректно утверждать, что агентные системы в целом не обладают внешней верификацией: отдельные работы, включая NOMAD и C4Modeler, уже содержат явные механизмы проверки. Более точное различие состоит в характере критерия и его включении в цикл работы. Проверка кода, синтаксиса UML или соблюдения правил C4 не эквивалентна детерминированной проверке инфраструктурного бюджета и ограничений времени ответа, очередей или хвостовой задержки по заданному профилю трафика.

Для исследуемого подхода принципиальна связка из трех частей: (1) LLM формирует или корректирует только архитектурное представление в заданной схеме; (2) обычные программные валидаторы проверяют структурные инварианты и предметные ограничения, не полагаясь на самооценку модели; (3) диагностические нарушения возвращаются на ограниченный шаг исправления. В таком описании мультиагентность не является обязательным условием метода: методологически важнее разделить вероятностную генерацию и детерминированное принятие решения о допустимости результата.

3.4 Теория массового обслуживания и модели производительности

Классическая теория массового обслуживания дает аналитические оценки загрузки, ожидания в очереди и времени пребывания заявки в системе [17,18]. Сети Джексона формализуют потоки заявок между несколькими узлами и допускают разложение стационарного распределения на узловые распределения при выполнении соответствующих условий: открытая сеть, пуассоновские внешние поступления, марковская маршрутизация и экспоненциальные времена обслуживания в стандартной модели [19]. Это позволяет анализировать сеть не как набор независимых диаграмм, а как систему взаимосвязанных очередей.

Для узла с m_i параллельными обслуживающими устройствами необходимым условием стационарности стандартной модели является

$$\rho_i = \frac{\lambda_i}{m_i \mu_i} < 1,$$

где λ_i — эффективная интенсивность поступления заявок, а μ_i — интенсивность обслуживания одного устройства. Для очередей с произвольными распределениями времени поступления и обслуживания применимы иные модели и приближения, в частности аппроксимация Кингмана для G/G/1 [20]. Нельзя без дополнительных условий одновременно приписывать произвольной сети M/G/m свойство произведения распределений, характерное для классической сети Джексона.

В контексте микросервисов аналитическая модель полезна для быстрой предварительной оценки и выявления узлов, приближающихся к насыщению. Однако ее результат зависит от предположений о распределениях, маршрутизации, количестве параллельных обработчиков и стационарности потока. Резкие всплески, fan-out, повторные попытки, ограниченные буферы, совместное использование CPU или пула соединений, autoscaling и корреляции между вызовами могут нарушать эти упрощения. Поэтому теория очередей не заменяет моделирование переходных процессов или измерения: аналитические оценки целесообразно использовать как отдельный уровень проверки, а пиковые и нестационарные сценарии исследовать симуляцией и, когда это возможно, сверять с нагрузочным стендом.

3.5 Сопоставление с исследуемым подходом

Таблица 1 суммирует не общие достоинства публикаций, а функции, непосредственно относящиеся к предмету исследования. Утверждения о возможностях аналогов ограничены функциями, описанными в соответствующих публикациях.

По рассмотренным источникам не следует заявлять, что ранее не существовало генерации диаграмм LLM, мультиагентного проектирования, проверки архитектурных моделей, имитации микросервисных систем или применения теории очередей: все эти направления имеют опубликованные аналоги. Более узкое и проверяемое позиционирование состоит в исследовании интегрированного контура: генерация типизированного графа микросервисной архитектуры, формальные правила допустимых связей, проверка NFR и диагностически направленный ремонт. Если симуляция пиковых нагрузок включена в этот контур, ее потенциальное отличие заключается в передаче выявленных узких мест обратно на этап коррекции топологии; эффективность такой связки должна оцениваться экспериментально.

4 Постановка задачи

Пусть сценарий проектирования задан кортежем:

$$\mathcal{S} = \langle \mathcal{D}, \mathcal{W}, \mathcal{R}_{NFR}, \mathcal{T}_{allowed} \rangle, \tag{1}$$

где $\mathcal{D}$ — текстовое описание предметной области; $\mathcal{W} = \langle \lambda_{base}, \lambda_{peak}, r_{read} \rangle$ — профиль входящей нагрузки (базовый RPS, пиковый RPS, доля операций чтения); $\mathcal{R}_{NFR} = \{R_1, \ldots, R_m\}$ — множество нефункциональных требований; $\mathcal{T}_{allowed} \subseteq \mathcal{T}$ — подмножество разрешенных типов узлов.

Синтезируемая архитектура формализуется в виде ориентированного графа $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, где:

- $\mathcal{V} = \{v_1, \ldots, v_n\}$ — множество функциональных узлов системы, $v_i = \langle id_i, type_i, repl_i, \mathbf{c}_i \rangle$, где $type_i \in \mathcal{T}_{allowed}$, $repl_i \geq 1$ — количество реплик узла, $\mathbf{c}_i$ — вектор емкостных характеристик (максимальная конкурентность C_{max}, лимит очереди Q_{max}, математическое ожидание μ_T и дисперсия σ_T времени обработки, ежемесячная стоимость $Cost_i$);
- $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ — множество направленных ребер взаимодействия $e = \langle src, dst, proto, \tau_{net} \rangle$, где $proto$ — протокол связи, τ_{net} — сетевая задержка.

Таблица 1: Сопоставление предшествующих подходов с исследуемым методом

Подход	**Что уже решается**	**Отличие относительно исследуемой задачи**
NOMAD [6]	Мультиагентная генерация UML-классов, промежуточная JSON-модель, отдельный агент-валидатор, классификация ошибок.	Предметная модель классов; в опубликованном описании нет проверки микросервисной топологии по профилям нагрузки и ремонтов на основе расчета производительности.
MAAD и C4Modeler [9, 10]	Ролевое проектирование архитектуры; генерация C4; структурные проверки и оценка архитектурного качества.	Основное внимание уделено созданию и оценке архитектурных артефактов; связанный цикл «нарушение NFR по модели нагрузки — локализация узкого места — коррекция топологии» в описанных методах не представлен.
PerfSim и CloudNativeSim [4, 11]	Дискретно-событийная симуляция микросервисных цепочек, оценка задержек и QoS, исследование вариантов ресурсов и политик масштабирования.	Симуляторы оценивают заданные модели/сценарии; сами по себе они не синтезируют требованиями управляемый архитектурный граф и не формируют LLM-предложения для исправления ошибок этого графа.
Palladio и модель Khazaei et al. [12, 13]	Параметрические модели компонентов, анализ ресурсов, симуляция и планирование ёмкости.	Подтверждают зрелость модельно-ориентированного анализа производительности, но не описывают связку LLM-генерации микросервисной топологии с направленным циклом исправления.
DeathStarBench [14]	Нагрузочные эксперименты на развернутых микросервисных приложениях и анализ сквозных эффектов.	Это эмпирический бенчмарк, а не генератор архитектур или симулятор; полезен для проверки внешней валидности предсказаний.
Исследуемый подход AutoSystemDesign	Структурированное представление архитектуры как графа; проверка ссылочной целостности, топологических правил и разрешенных типов связей; ограничение бюджета и проверка NFR; направленный режим INITIAL/REPAIR с передачей диагностик; экспорт в C4 Container.	Исследуется интеграция генерации и детерминированной верификации в замкнутый контур. Целевой уровень развития метода — использование количественной оценки производительности, включая моделирование очередей и нестационарной нагрузки, для локализации нарушений и корректировки топологии.

Задача состоит в нахождении такого графа $\mathcal{G}^*$, который:

1. Строго удовлетворяет структурной архитектурной онтологии: $\text{Valid}_{struct}(\mathcal{G}^*) = \textbf{true}$;
2. Удовлетворяет всем нефункциональным ограничениям: $\forall R \in \mathcal{R}_{NFR} : R(\mathcal{G}^*, \mathcal{W}) = \textbf{true}$;
3. Минимизирует суммарную инфраструктурную стоимость: $\sum_{v_i \in \mathcal{V}} Cost(v_i) \to \min$.

5 Метод оптимизации архитектур в замкнутом контуре

5.1 Двухфазный цикл: генерация (`INITIAL`) и направленное исправление (`REPAIR`)

Для предотвращения накопления ошибок в ходе длинных контекстных диалогов в разработанном методе четко разделены режимы вызова генеративного агента:

$$Mode \in \{\texttt{INITIAL}, \texttt{REPAIR}\}. \tag{2}$$

На шаге $k = 1$ формируется задача $Task_{initial} = \langle \mathcal{S}, \texttt{INITIAL} \rangle$. Агент генерирует первоначальную структуру графа на основе входного сценария, сокращенной схемы целевого формата и демонстрационного примера (few-shot).

Если сгенерированный результат содержит синтаксические повреждения или нарушает системные инварианты, управление передается в контур исправления. Максимальное число шагов ограничено параметром $k_{max} \in [1, 10]$ (в базовой конфигурации $k_{max} = 3$). На шаге $k > 1$ агент решает задачу ремонта:

$$Task_{repair} = \langle \mathcal{S}, \texttt{REPAIR}, Payload_{k-1}, \mathcal{F}_{parse}, \mathcal{F}_{valid} \rangle, \tag{3}$$

где $Payload_{k-1}$ — сырой ответ предыдущей итерации; $\mathcal{F}_{parse}$ — детализированный отчет синтаксических ошибок JSON; $\mathcal{F}_{valid}$ — формализованный список архитектурных нарушений.

5.2 Диагностическая обратная связь при ошибках JSON Structured Output

В отличие от наивных подходов, где модель получает лишь общее сообщение «JSON неверен, попробуй еще раз», в предложенном алгоритме парсер `parse_architecture_graph` формирует список точечных объектов `ProposalFailureDetail`.

Каждая ошибка локализуется в JSONPath-нотации:

$$Location = \texttt{\$.nodes[i].maximum_concurrency}, \quad ErrorType = \texttt{missing / type_error}. \tag{4}$$

Если модель выдает невалидный JSON (например, обрезанный из-за лимита токенов или содержащий синтаксические ошибки), фиксируется тип `json_decode` с точным указанием символа и строки сбоя. В промпт исправления инжектируется нумерованный список обнаруженных дефектов, что позволяет LLM точечно скорректировать именно сбойные поля графа без полной деградации остальной структуры.

5.3 Конвейер валидаторов

Процедура валидации спроектирована по паттерну «Компоновщик / Цепочка обязанностей» как последовательность independent валидаторов:

$$\mathcal{P}_{validation} = \langle \text{Validator}_1, \text{Validator}_2, \ldots, \text{Validator}_m \rangle. \tag{5}$$

Все валидаторы выполняются последовательно над единым представлением графа $\mathcal{G}$, а их ошибки агрегируются в единый контейнер `ValidationResult`. Данная архитектура позволяет бесшовно подключать новые виды проверок (структурные, ТМО-аналитические, симуляционные) без изменения кода службы оркестрации `DesignGenerationService`.

6 Архитектурная онтология и топологический валидатор

6.1 Правила топологической корректности

Валидатор `StructuralTopologyValidator` реализует строгий набор предикатов первого порядка:

1. **Уникальность идентификаторов**: $\forall u, v \in \mathcal{V} : (u \neq v) \implies (id(u) \neq id(v))$.
2. **Ссылочная целостность ребер**: $\forall e = (src, dst) \in \mathcal{E} : (src \in \mathcal{V}) \wedge (dst \in \mathcal{V})$.
3. **Отсутствие изолированных вершин**: В многовершинных графах ($|\mathcal{V}| > 1$) ни один узел не должен иметь нулевую полную степень связности: $\forall v \in \mathcal{V} : \deg^+(v) + \deg^-(v) \geq 1$.
4. **Строгая ацикличность (DAG)**: Поскольку синхронные межсервисные циклы вызывают распределенные взаимоблокировки и бесконечные каскады вызовов, граф взаимодействия обязан быть ациклическим. Проверка осуществляется за линейное время $O(|\mathcal{V}| + |\mathcal{E}|)$ с помощью алгоритма поиска в глубину с трехцветной раскраской вершин (0 — unvisited, 1 — in recursion stack, 2 — visited).

6.2 Матрица разрешенных межсервисных переходов

Для исключения архитектурных антипаттернов над графом задана онтология допустимых связей между классами компонентов:

$$\mathcal{T}_{valid} \subset \mathcal{T}_{source} \times \mathcal{T}_{target}. \tag{6}$$

В базовый белый список `DEFAULT_ALLOWED_CONNECTION_TYPES` входят исключительно безопасные переходы:

- Входной трафик: `CLIENT` → `LOAD_BALANCER`, `CLIENT` → `COMPUTE`;
- Балансировка: `LOAD_BALANCER` → `COMPUTE`;
- Внутренняя обработка: `COMPUTE` → `COMPUTE`, `COMPUTE` → `DATABASE`, `COMPUTE` → `CACHE`, `COMPUTE` → `MESSAGE_BROKER`;
- Асинхронные воркеры: `MESSAGE_BROKER` → `COMPUTE`.

Попытка модели связать `CLIENT` → `DATABASE` пресекается ошибкой `DISALLOWED_CONNECTION_TYPE`: внешний клиент не имеет права прямого доступа к хранилищу. Аналогично, обратная связь `DATABASE` → `LOAD_BALANCER` недопустима, так как слой хранения не должен обладать знанием о входной сетевой маршрутизации.

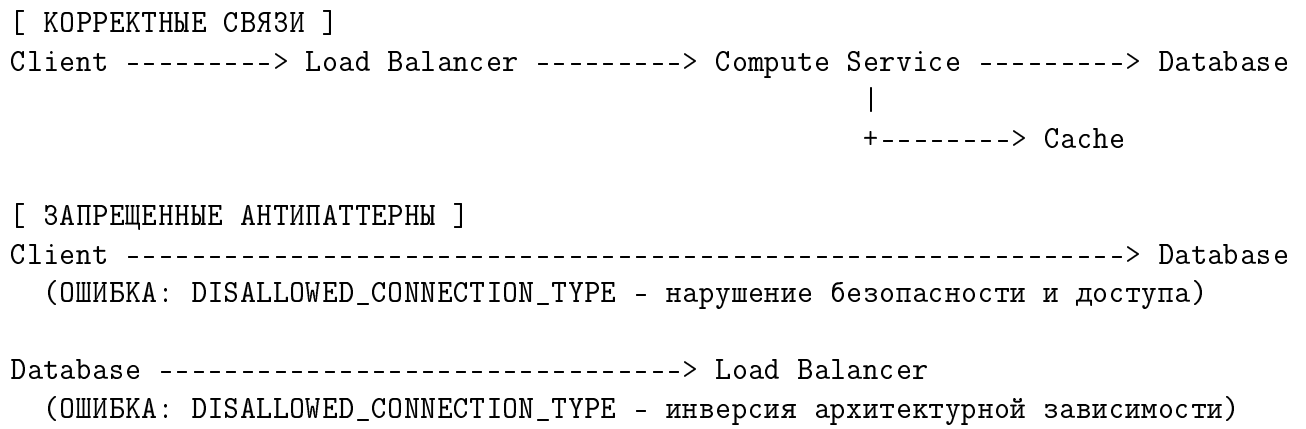


Рис. 1: Схематическое сопоставление допустимых архитектурных связей и выявляемых антипаттернов

7 Доменная модель расширяемости нефункциональных требований

Для формализации разнообразных нефункциональных требований (NFR) разработана иерархия классов в модуле `domain.models.base_constraints`.

1. **Односторонние пороговые ограничения** (`ThresholdRequirement`): Задаются оператором сравнения $\odot \in \{<, \leq, ==, \geq, >\}$ и порогом $\theta \in \mathbb{R}$:

$$\text{IsMet}(x) \iff x \odot \theta. \tag{7}$$

   Для устранения погрешностей машинной точности оператор равенства $==$ реализован через интервальное приближение $|x - \theta| \leq \varepsilon$.

2. **Двусторонние интервальные ограничения** (`IntervalRequirement`): Контролируют попадание в технологический коридор $[V_{min}, V_{max}]$:

$$\text{IsMet}(x) \iff (V_{min} \leq x \leq V_{max}). \tag{8}$$

3. **Перцентильные ограничения** (`PercentileThresholdRequirement`): Расширяют пороговые требования вероятностным перцентилем $P \in [0, 100]$ (например, 99-й перцентиль задержки $p_{99} \leq 300$ мс).

Благодаря механизму дискриминированных объединений (Discriminated Unions в Pydantic) новые виды требований (бюджет, задержка критического пути, максимальный размер очереди, процент доступности) регистрируются через строковый идентификатор `type_identifier`, исключая потерю полей при десериализации.

8 Валидация бюджета и профили ресурсов инфраструктуры

Валидатор бюджета суммирует расходы на аренду всех вычислительных емкостей в топологии:

$$Cost_{total}(\mathcal{G}) = \sum_{v \in \mathcal{V}_{capacity}} repl(v) \cdot MonthlyCost(v). \tag{9}$$

Если $Cost_{total}(\mathcal{G}) > Budget_{max}$, генерируется доменная ошибка превышения бюджета с указанием дельты перерасхода.

Характеристики и стоимость серверов формируются на основе открытых данных каталогов публичных облачных провайдеров (например, AWS EC2, Amazon RDS, ElastiCache, Amazon SQS). В табл. 2 приведены типовые профили компонентов, зафиксированные в реестре `ProfileRegistry`.

Таблица 2: Примеры профилей аппаратных ресурсов и компонентов на базе тарифов облачных провайдеров

Класс узла	**Аналог AWS**	**Макс. конк.** (C_{max})	**Время обр.** $\mu \pm \sigma$ (мс)	**IOPS**	**Стоимо** ($/мес
`LOAD_BALANCER`	ALB / NLB	5000	1.0 ± 0.2	—	$22.5
`COMPUTE`	c6g.xlarge (4 vCPU, 8GB)	250	15.0 ± 3.0	—	$98.5
`COMPUTE`	c6g.4xlarge (16 vCPU)	1000	10.0 ± 2.0	—	$394.0
`DATABASE`	db.r6g.xlarge (PostgreSQL)	150	8.0 ± 2.0	3000	$260.0
`CACHE`	cache.m6g.large (Redis)	2000	0.8 ± 0.2	—	$68.0
`MESSAGE_BROKER`	Amazon MSK / SQS	10000	2.0 ± 0.5	—	$150.0

9 Программная реализация комплекса AutoSystemDesign

Программный комплекс `AutoSystemDesign` спроектирован в соответствии с принципами чистой архитектуры (Clean Architecture) и DDD:

- **Слой домена** (`src/domain`): содержит бескомпромиссно чистые модели (узлы, ребра, сценарии), правила валидации и исключения предметной области; не зависит от внешних библиотек инференса.
- **Слой приложения** (`src/application`): инкапсулирует порты (`ArchitectureDesigner`), DTO задач и результатов, службу координации и итеративного восстановления (`DesignGenerati`
- **Слой инфраструктуры** (`src/infrastructure`): реализует адаптеры взаимодействия с LLM через библиотеку `litellm`, специализированный отказоустойчивый парсер структурированных данных и модуль визуализации `visualisation.plantuml`.

При получении валидного графа компонент `C4DiagramService` выполняет трансляцию структуры во внутреннее представление PlantUML C4 Container, после чего через сжатие raw DEFLATE и 64-символьное кодирование URL генерирует автономные векторные схемы (SVG/PNG) без необходимости локальной установки среды Java.

10 Заключение

В работе представлен научно обоснованный метод и архитектура программного комплекса `AutoSystemDesign` для генерации и верификации архитектур программных комплексов. За счет объединения гибкости больших языковых моделей с детерминированным конвейером проверки топологических правил и ограничений нефункциональных требований предложенный подход устраняет проблему архитектурных галлюцинаций. Наличие детализированной обратной связи на шагах `REPAIR` обеспечивает целенаправленную сходимость графа к валидному состоянию при минимальных накладных расходах по токенам.

Список литературы

[1] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*. Addison-Wesley Professional, 3rd ed., 2015.

[2] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, pp. 1–79, 2024.

[3] J. Liu, K. Wang, Y. Chen, X. Peng, Z. Chen, L. Zhang, and Y. Lou, "Large language model-based agents for software engineering: A survey," *arXiv preprint arXiv:2409.02977*, 2024.

[4] M. Gokan Khan, J. Taheri, A. Al-Dulaimy, and A. Kassler, "Perfsim: A performance simulator for cloud native microservice chains," *IEEE Transactions on Cloud Computing*, vol. 11, no. 2, pp. 1395–1413, 2023.

[5] S. Brown, *Software Architecture for Developers: Visualise, Document and Explore Your Software Architecture*. Leanpub, 2018.

[6] P. Giannouris and S. Ananiadou, "Nomad: A multi-agent llm system for uml class diagram generation from natural language requirements," *arXiv preprint arXiv:2511.22409*, 2025.

[7] A. Ferrari, S. Abualhaija, and C. Arora, "Model generation with llms: From requirements to uml sequence diagrams," in *Proceedings of the 32nd IEEE International Requirements Engineering Conference Workshops (REW 2024)*, pp. 291–300, 2024.

[8] M. Megahed, "Archigen: An llm-augmented framework for automated generation of software solution blueprint documents from uml diagrams," in *2026 Intelligent Methods, Systems, and Applications (IMSA)*, IEEE, 2026.

[9] R. Li, Y. Zhang, X. Zhou, P. Liang, W. Sun, J. Xuan, Z. Jin, and Y. Liu, "Maad: Automate software architecture design through knowledge-driven multi-agent collaboration," *arXiv preprint arXiv:2507.21382*, 2025.

[10] K. Szczepanik and J. A. Chudziak, "Collaborative llm agents for c4 software architecture design automation," *arXiv preprint arXiv:2510.22787*, 2025.

[11] J. Wu, M. Xu, Y. He, K. Ye, and C. Xu, "Cloudnativesim: A toolkit for modeling and simulation of cloud-native applications," *arXiv preprint arXiv:2409.05093*, 2024.

[12] S. Becker, H. Koziolek, and R. H. Reussner, "The palladio component model for model-driven performance prediction," *Journal of Systems and Software*, vol. 82, no. 1, pp. 3–22, 2009.

[13] H. Khazaei, N. Mahmoudi, C. Barna, and M. Litoiu, "Performance modeling of microservice platforms," *IEEE Transactions on Cloud Computing*, vol. 10, no. 4, pp. 2848–2862, 2022.

[14] Y. Gan, Y. Zhang, D. Cheng, A. Shetty, P. Rathi, N. Katarki, A. Bruno, C. Hu, B. Ritchken, B. Jackson, *et al.*, "An open-source benchmark suite for microservices and their hardware-software implications for cloud and edge systems," in *Proceedings of the 24th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 3–18, 2019.

[15] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "Chatdev: Communicative agents for software development," *arXiv preprint arXiv:2307.07924*, 2023.

[16] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "Metagpt: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations (ICLR)*, 2024.

[17] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. John Wiley & Sons, 1975.

[18] M. Harchol-Balter, *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge University Press, 2013.

[19] J. R. Jackson, "Networks of waiting lines," *Operations Research*, vol. 5, no. 4, pp. 518–521, 1957.

[20] J. F. C. Kingman, "The single server queue in heavy traffic," *Mathematical Proceedings of the Cambridge Philosophical Society*, vol. 57, no. 4, pp. 902–904, 1961.